

# Transformers in Diffusion Models for Image Generation and Beyond



Stanford, CS25, May'25

Sayak Paul

Hugging Face 🤗

# Disclaimer

- This talk is NOT an exhaustive overview of all possible methods.
- Will not cover what is diffusion or flow-matching in details.
- Architectures will be agnostic to diffusion or flow.
- Will take examples with images, primarily.
- I will share the slides after the talk.

# Overview

- Diffusion models, briefly
- Early architectures for diffusion-based image generation
- DiTs and friends
- Concluding thoughts
- QnA

# Era of text-to-image diffusion models!



*"A small cactus with a happy face in the Sahara desert."*

**PixArt-Alpha**



*"panda mad scientist mixing sparkling chemicals, digital art."*

**DALL-E 3**



*"A tiny astronaut hatching from an egg on the moon"*

**Flux.1 Merged**

# Diffusion models in a jiffy

What happens when you refine a noise vector to become a realistic image?

Data

Noise

<https://nvlabs.github.io/denoising-diffusion-gan/>

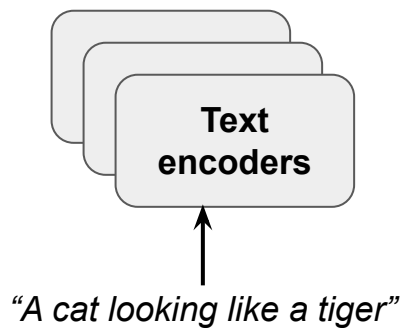
# Diffusion models in a jiffy

When you “condition” the denoising process with text:

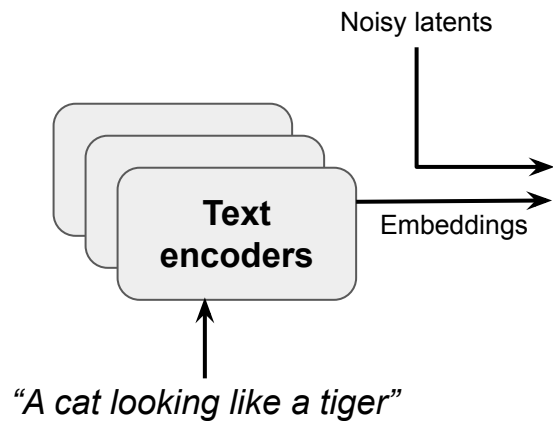


Flux prompt: “A photo of a white fur monster standing in a purple room”

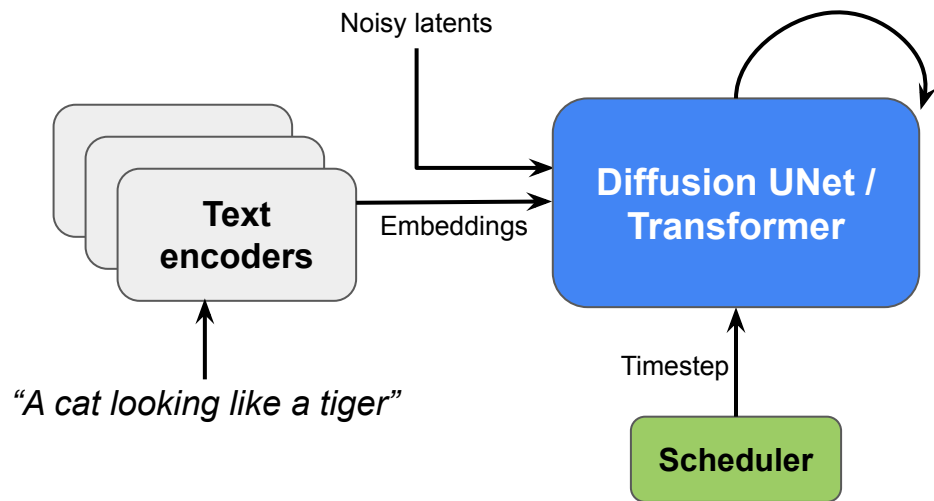
# Diffusion models are NOT single models



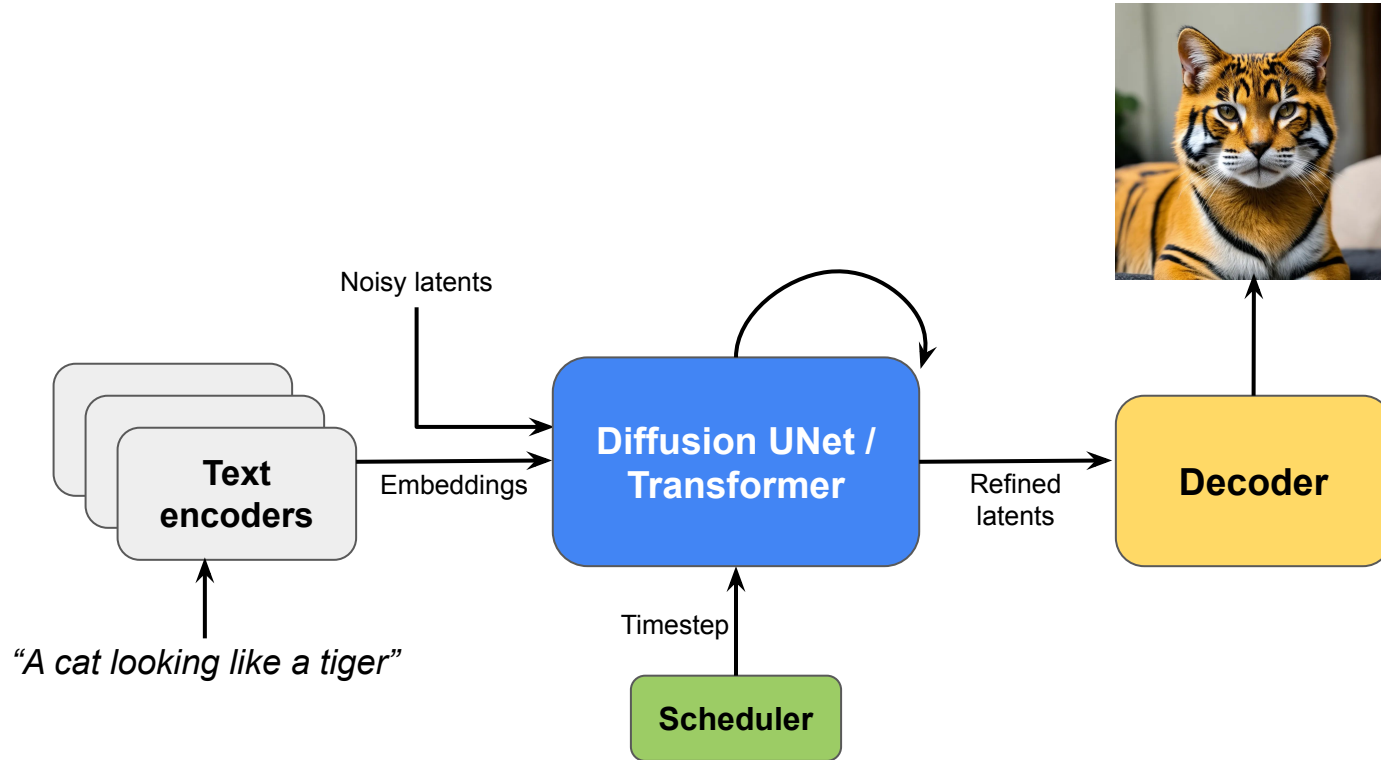
# Diffusion models are NOT single models



# Diffusion models are NOT single models



# Diffusion models are NOT single models



# Diffusion models in a jiffy

Without the loss of generality, let:

- $\mathbf{x}$ : original image
- $\boldsymbol{\epsilon}$ : noise;  $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
- $t$ : diffusion process time;  $t \sim \mathcal{U}([0, 1])$
- $\alpha_t, \sigma_t, \mathbf{w}_t$ : terms controlling noise schedule and sample quality
- $\mathbf{c}$ : conditioning vector (prompt embeddings, for example)
- $\hat{\mathbf{x}}_\theta$ : diffusion model to be learned

*Training*  $\mathbb{E}_{\mathbf{x}, \mathbf{c}, \boldsymbol{\epsilon}, t} \left[ w_t \|\hat{\mathbf{x}}_\theta(\alpha_t \mathbf{x} + \sigma_t \boldsymbol{\epsilon}, \mathbf{c}) - \mathbf{x}\|_2^2 \right]$

*Inference*  $\mathbf{x}_{\text{gen}} = \hat{\mathbf{x}}_\theta(\boldsymbol{\epsilon}, \mathbf{c})$

# Diffusion models in a jiffy

For flow-matching, this becomes:

- $\mathbf{x}_t = (1 - t)\mathbf{x}_0 + t\boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
- With a velocity field:  $\mathbf{u}(\mathbf{x}_t, t) = \boldsymbol{\epsilon} - \mathbf{x}_0$
- $\hat{\mathbf{u}}_\theta(\mathbf{x}_t, t)$  is trained to approximate this target velocity

# Diffusion models in a jiffy



Focus of this talk

$$\mathbb{E}_{\mathbf{x}, \mathbf{c}, \epsilon, t} \left[ w_t \left\| \hat{\mathbf{x}}_{\theta}(\alpha_t \mathbf{x} + \sigma_t \epsilon, \mathbf{c}) - \mathbf{x} \right\|_2^2 \right]$$

Scheduler math

# A diffusion model - the components

- Deal with the (noisy) inputs:
  - Pixel-space  $(B, 3, H, W)$
  - Latent-space  $(B, \text{latent\_channels}, H // \text{scale\_factor}, W // \text{scale\_factor})$
- Deal with the conditions:
  - Timestep
  - Class, text, etc.
- Model the dependencies:
  - How should the noisy inputs interact with the conditions?
- How the final outputs be produced? Upsampling? Decoding?
- Usually, `input.shape == output.shape`

# A diffusion model - the components

- The early models (DDPM and LDM) used a UNet-based architecture.
- Historically speaking, UNets have always dominated this kind of space for images.
- So, a natural choice for the diffusion network has been UNet-based till SDXL.

Denoising Diffusion Probabilistic Models (DDPM)

High-Resolution Image Synthesis with Latent Diffusion Models (LDM)

SDXL: Improving Latent Diffusion Models for High-Resolution Image Synthesis (SDXL)

# A diffusion model - the giant UNet

- An input convolution stem
- A series of downblocks
  - Custom residual blocks (comprising normalization, convolutions, etc.)
  - Custom transformer blocks (comprising normalization, projections, and regular transformers)
  - Convolution layers for upsampling
- A single midblock (similar but no resolution changes)
- A series of upblocks
  - Similar as downblocks with convolution layers for downsampling to match the input resolution
- Misc:
  - Timestep embeddings\*
  - Additional embeddings (class embeddings, micro-conditions, text embeddings)

\*Is Noise Conditioning Necessary for Denoising Generative Models?

# A diffusion model - the giant UNet 🙄

```
self.time_embeddings = nn.Sequential(
    SinusoidalTimescaleEmbeddings(320),
    nn.Linear(in_features=320, out_features=1280),
    nn.SiLU(),
    nn.Linear(1280, 1280),
)

residuals = [UNetResidual() for _ in range(12)]

# fnt: off

self.down_blocks = nn.Sequential(
    # 4 -> 320
    nn.Conv2d(1, in_channels=4, out_channels=320, kernel_size=3, padding=1), residuals[0],

    # 320 -> 320
    UnetResNetBlock2D(320, 320), TransformerDecoder2D(320), residuals[1],
    UnetResNetBlock2D(320, 320), TransformerDecoder2D(320), residuals[2],
    nn.Conv2d(320, 320, kernel_size=3, stride=2, padding=1), residuals[3],

    # 320 -> 640
    UnetResNetBlock2D(320, 640), TransformerDecoder2D(640), residuals[4],
    UnetResNetBlock2D(640, 640), TransformerDecoder2D(640), residuals[5],
    nn.Conv2d(640, 640, kernel_size=3, stride=2, padding=1), residuals[6],

    # 640 -> 1280
    UnetResNetBlock2D(640, 1280), TransformerDecoder2D(1280), residuals[7],
    UnetResNetBlock2D(1280, 1280), TransformerDecoder2D(1280), residuals[8],
    nn.Conv2d(1280, 1280, kernel_size=3, stride=2, padding=1), residuals[9],

    # 1280 -> 1280
    UnetResNetBlock2D(1280, 1280), residuals[10],
    UnetResNetBlock2D(1280, 1280), residuals[11],
)

self.mid_block = nn.Sequential(
    UnetResNetBlock2D(1280, 1280), TransformerDecoder2D(1280), UnetResNetBlock2D(1280, 1280)
)

self.up_blocks = nn.Sequential(
    # 1280 -> 1280
    residuals[11], UnetResNetBlock2D(1280 + 1280, 1280),
    residuals[10], UnetResNetBlock2D(1280 + 1280, 1280),
    residuals[9], UnetResNetBlock2D(1280 + 1280, 1280),
    nn.Upsample(LingNearst2d(scale_factor=2.0), nn.Conv2d(1280, 1280, kernel_size=3, padding=1),

    # 1280 -> 640
    residuals[8], UnetResNetBlock2D(1280 + 1280, 1280), TransformerDecoder2D(1280),
    residuals[7], UnetResNetBlock2D(1280 + 1280, 1280), TransformerDecoder2D(1280),
    residuals[6], UnetResNetBlock2D(1280 + 1280, 1280), TransformerDecoder2D(1280),
    nn.Upsample(LingNearst2d(scale_factor=2.0), nn.Conv2d(1280, 1280, kernel_size=3, padding=1),

    # 640 -> 320
    residuals[5], UnetResNetBlock2D(640 + 1280, 640), TransformerDecoder2D(640),
    residuals[4], UnetResNetBlock2D(640 + 1280, 640), TransformerDecoder2D(640),
    residuals[3], UnetResNetBlock2D(640 + 1280, 640), TransformerDecoder2D(640),
    nn.Upsample(LingNearst2d(scale_factor=2.0), nn.Conv2d(640, 640, kernel_size=3, padding=1),

    # 320 -> 320
    residuals[1], UnetResNetBlock2D(320 + 640, 320), TransformerDecoder2D(320),
    residuals[11], UnetResNetBlock2D(320 + 320, 320), TransformerDecoder2D(320),
    residuals[10], UnetResNetBlock2D(320 + 320, 320), TransformerDecoder2D(320),

    # 320 -> 4
    nn.GroupNorm(num_groups=32, num_channels=320),
    nn.SiLU(),
    nn.Conv2d(320, 4, kernel_size=3, padding=1),
)
```

<https://x.com/williamLberman/status/1679380255671459841>

# Giant UNet - downblocks

```
1 nn.ModuleList([
2     # 320 -> 320
3     nn.ModuleDict(dict(
4         resnets=nn.ModuleList([
5             ResnetBlock2D(320, 320, time_embedding_dim),
6             ResnetBlock2D(320, 320, time_embedding_dim),
7         ]),
8         downsamplers=nn.ModuleList([nn.ModuleDict(dict(conv=nn.Conv2d(320, 320,
9             kernel_size=3, stride=2, padding=1))))]),
10    ]),
11    # 320 -> 640
12    nn.ModuleDict(dict(
13        resnets=nn.ModuleList([
14            ResnetBlock2D(320, 640, time_embedding_dim),
15            ResnetBlock2D(640, 640, time_embedding_dim),
16        ]),
17        attentions=nn.ModuleList([
18            TransformerDecoder2D(640, encoder_hidden_states_dim,
19                num_transformer_blocks=2),
20            TransformerDecoder2D(640, encoder_hidden_states_dim,
21                num_transformer_blocks=2),
22        ]),
23        downsamplers=nn.ModuleList([nn.ModuleDict(dict(conv=nn.Conv2d(640, 640,
24            kernel_size=3, stride=2, padding=1))))]),
25    ]),
26    # 640 -> 1280
27    nn.ModuleDict(dict(
28        resnets=nn.ModuleList([
29            ResnetBlock2D(640, 1280, time_embedding_dim),
30            ResnetBlock2D(1280, 1280, time_embedding_dim),
31        ]),
32        attentions=nn.ModuleList([
33            TransformerDecoder2D(1280, encoder_hidden_states_dim,
34                num_transformer_blocks=10),
35            TransformerDecoder2D(1280, encoder_hidden_states_dim,
36                num_transformer_blocks=10),
37        ]),
38    ]),
39 ])
```

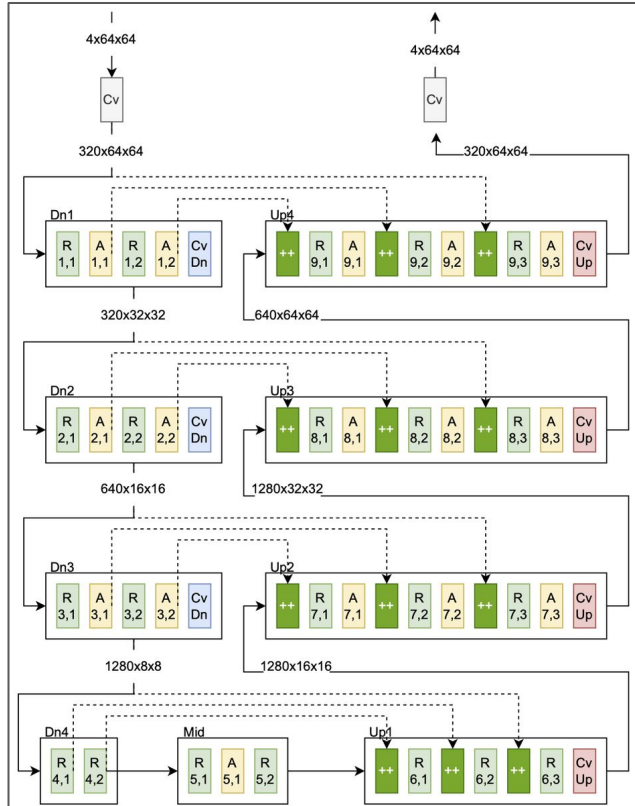
All are custom blocks:

ResNetBlock2D,

TransformerDecoder2D

<https://github.com/williamberman/nano-diffusion/blob/main/models.py>

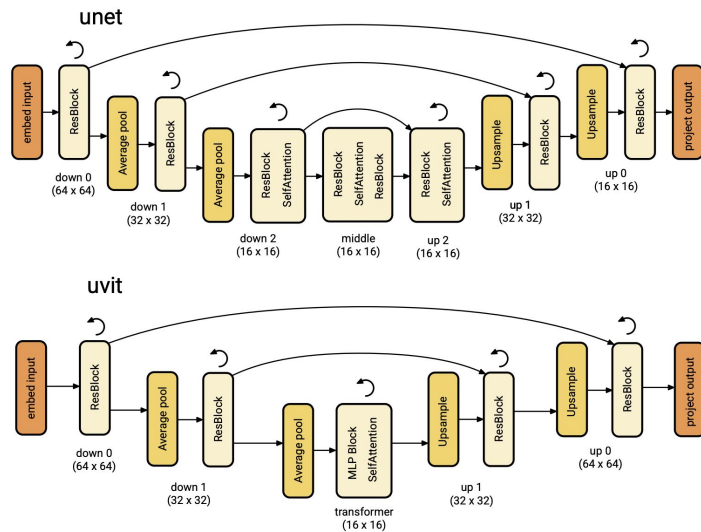
# Giant UNet - the complete perspective



<https://x.com/krasul/status/1679386250048831488>

# Marrying UNet + Transformer - UViT

- Replace convolutional layers with MLP blocks
- No skip connections in the middle block (only residual connections)



Simple diffusion: End-to-end diffusion for high resolution images

## Why bother about changing to DiT?

- Advancements in transformers benefit DiTs too (parallel-MLP, QK-Norm, GQA, fancy positional encoding, etc.).
- Known for good scaling properties.
- Allows for clean integration with other backbones (more on this later).
- Gets rid of the giant UNet.

# The OG DiT - site to the sore eyes!

Familiar with this pattern (a classic ViT forward pass):

```
1 def forward(self, x):
2     """
3     x: (N, C, H, W) tensor of spatial inputs (images or
4         latent representations of images)
5     """
6     x = self.patch_embed(x)
7     x = self._pos_embed(x)
8     x = self.patch_drop(x)
9     x = self.norm_pre(x)
10    for block in self.transformer_blocks:
11        x = block(x)
12    x = self.norm(x)
13    # x = ... # task-specific compute
14    return x
```

[Timm ViT](#)

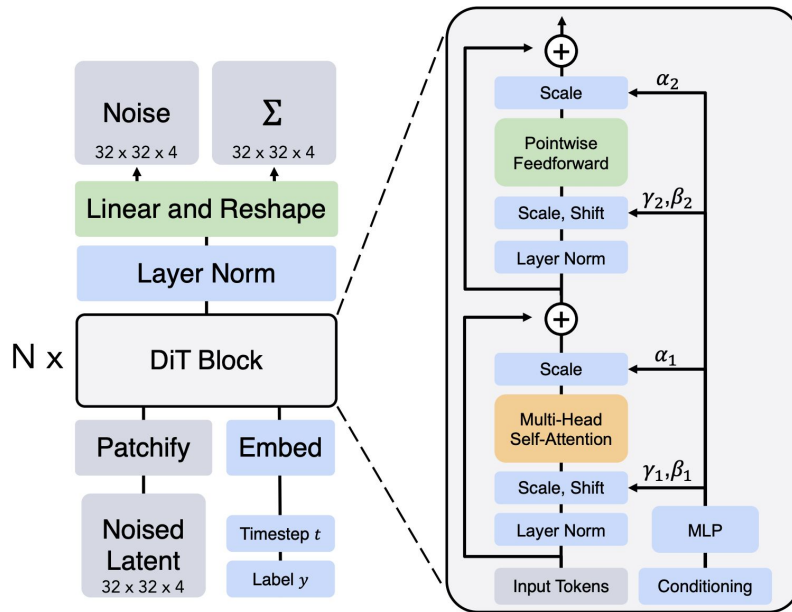
# The OG DiT - site to the sore eyes!

Not needed to change a lot for class-conditional image gen:

```
1 def forward(self, x, t, y):
2     """
3     x: (N, C, H, W) tensor of spatial inputs (images
4         or latent representations of images)
5     t: (N,) tensor of diffusion timesteps
6     y: (N,) tensor of class labels
7     """
8     # (N, T, D), where T = H * W / patch_size ** 2
9     x = self.x_embedder(x) + self.pos_embed
10    # (N, D)
11    t = self.t_embedder(t)
12    y = self.y_embedder(y, self.training)
13    # (N, D)
14    c = t + y
15    for block in self.blocks:
16        # (N, T, D)
17        x = block(x, c)
18    # (N, T, patch_size ** 2 * out_channels)
19    x = self.final_layer(x, c)
20    # (N, out_channels, H, W)
21    x = self.unpatchify(x)
22    return x
```

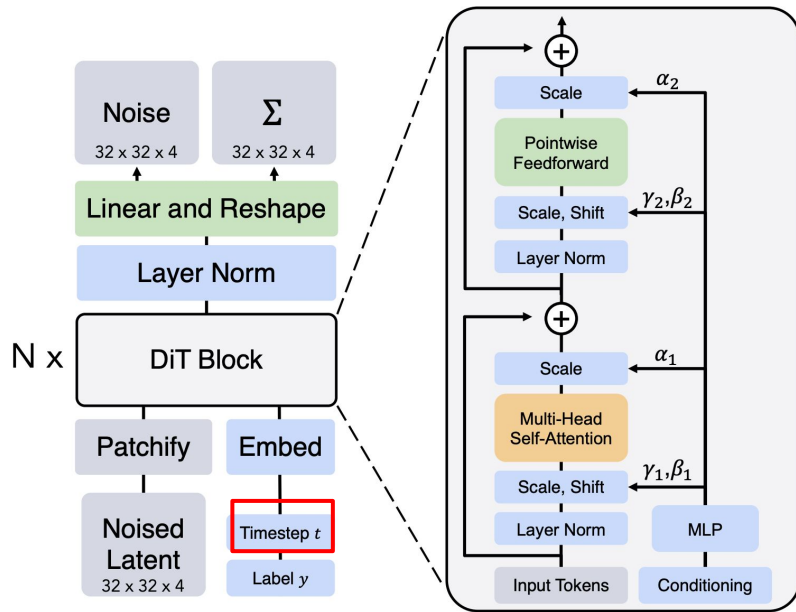
[Original DiT](#)

# The OG DiT - closer look



[Scalable Diffusion Models with Transformers](#)

# The OG DiT - closer look

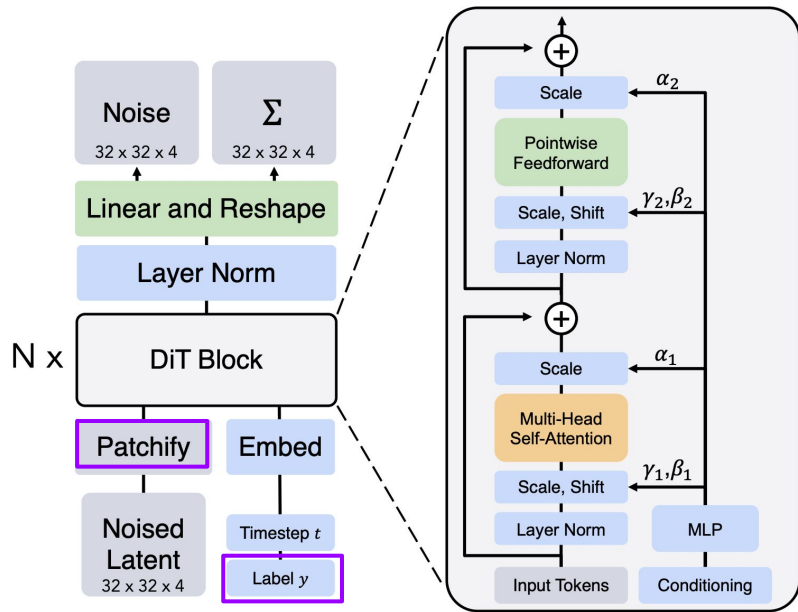


## Embedding **timesteps**:

- Timestep ranges from  $[0, 1000]$ , denoting the noise-level.
- Each  $t$  is embedded into sinusoidal frequency space (phase-awareness).
- Then passes through a 2-layer non-linear (SiLU) MLP.

$(B, D)$

# The OG DiT - closer look



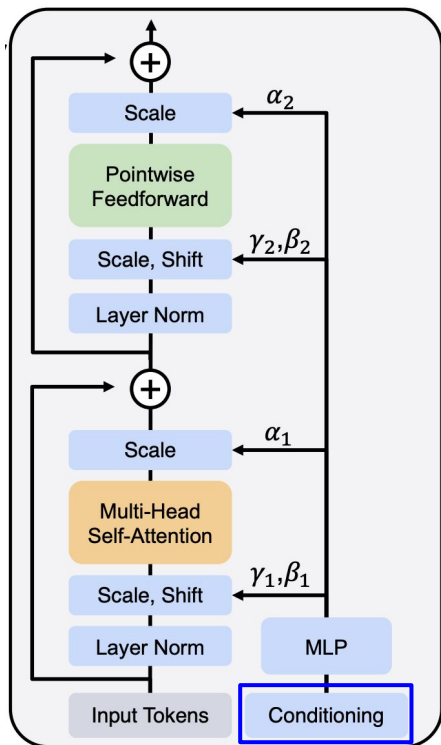
Embedding **class-labels**:

- Standard `nn.Embedding` layer with a dimensionality of DiT hidden size.
- $(B, D)$

Standard **patchification** + sin-cos pos embeddings  $(B, N, D)$

$$N = H * W / \text{patch\_size} ** 2$$

# The OG DiT - closer look



The **conditioning**:

$$\mathbf{t}_{\text{emb}} = \mathbf{t}_{\text{embed}}(t)$$

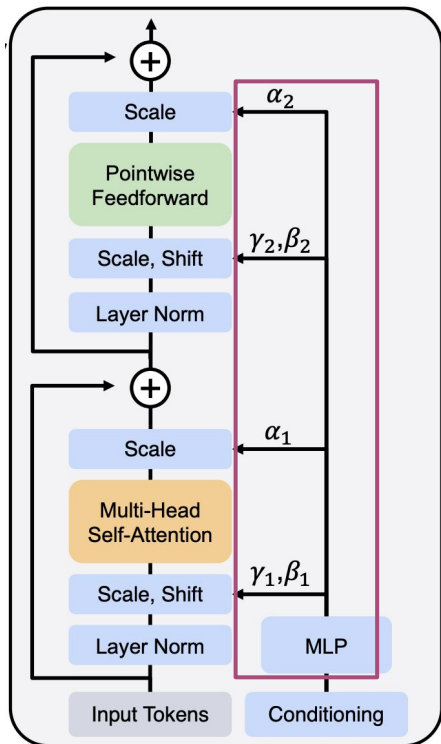
$$\mathbf{y}_{\text{emb}} = \mathbf{y}_{\text{embed}}(y)$$

$$\mathbf{c} = \mathbf{t}_{\text{emb}} + \mathbf{y}_{\text{emb}}$$

$(B, D)$

$\mathbf{c}$  remain **fixed** across blocks.

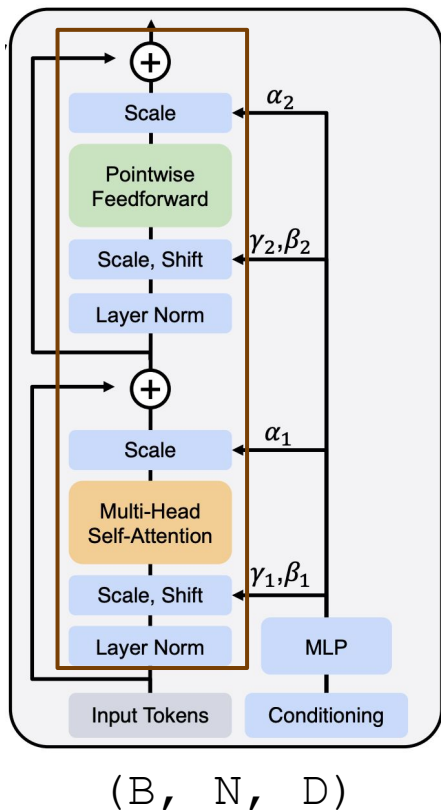
# The OG DiT - closer look



For each DiT block, **adaptive layer-norm** is regressed from the combined condition ( $c = y + t$ ).

```
1 adaLN = nn.Sequential(  
2     nn.SiLU(),  
3     nn.Linear(hidden_size, 6 * hidden_size, bias=True)  
4 )  
5 gamma_msa, beta_msa, alpha_msa, gamma_mlp, beta_mlp, alpha_mlp = (  
6     adaLN(c).chunk(6, dim=1)  
7 )
```

# The OG DiT - closer look



Skipping the usual transformer stuff (QKV, MSA, MLP), the governing equations:

$$\mathbf{x} \leftarrow \mathbf{x} + \text{Attn}(\text{LayerNorm}_1(\mathbf{x}))$$

$$\mathbf{x} \leftarrow \mathbf{x} + \text{MLP}(\text{LayerNorm}_2(\mathbf{x}))$$

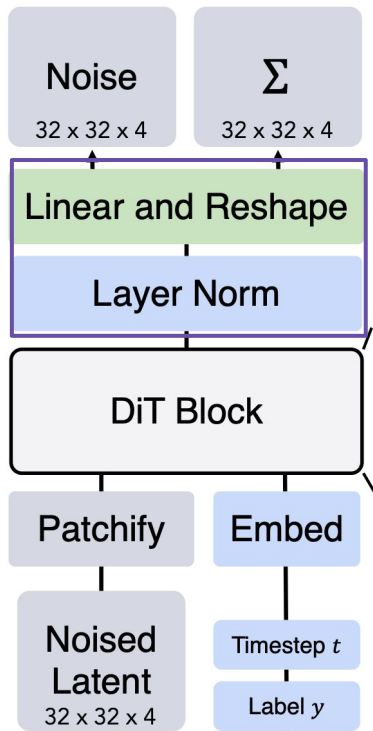
But for a **DiT**,

$$\mathbf{x} \leftarrow \mathbf{x} + \alpha_{\text{msa}} \text{Attn}(\text{modLN}_1(\mathbf{x}, \beta_{\text{msa}}, \gamma_{\text{msa}}))$$

$$\mathbf{x} \leftarrow \mathbf{x} + \alpha_{\text{mlp}} \text{MLP}(\text{modLN}_2(\mathbf{x}, \beta_{\text{mlp}}, \gamma_{\text{mlp}}))$$

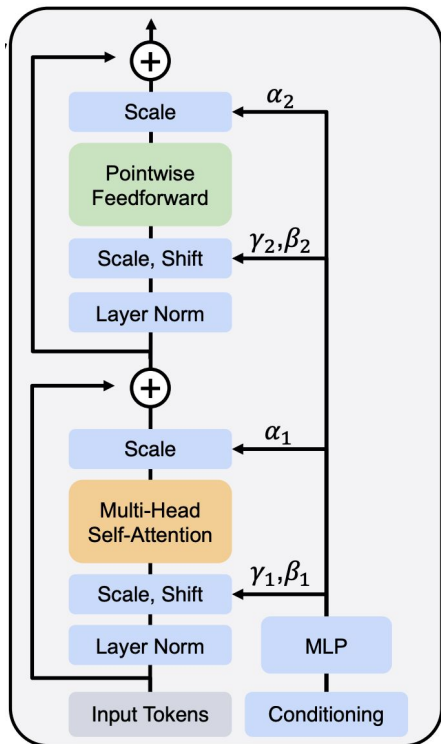
$$\text{modLN}_i(\mathbf{x}, \beta, \gamma) = (\text{LayerNorm}_i(\mathbf{x})) \odot (1 + \gamma) + \beta$$

# The OG DiT - closer look



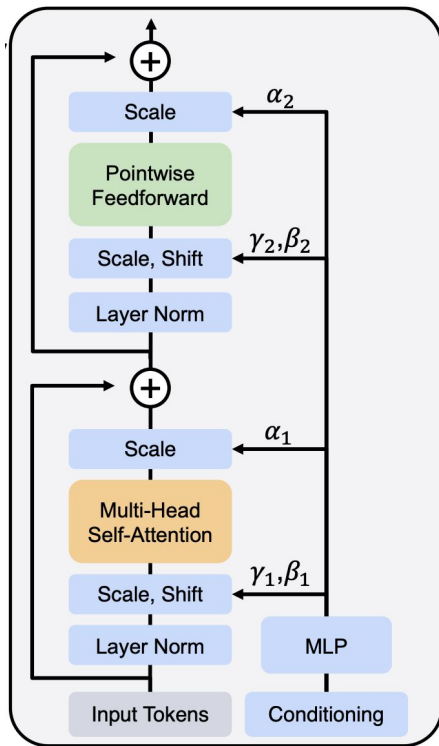
- **Single layer** to decode  $(B, N, D)$  into  $(B, N, (\text{patch\_size} ** 2) * \text{out\_channels})$
- **Reshape** into  $(B, \text{out\_channels}, H, W)$

# The OG DiT - a note on init



- Standard ViT init but ...
- Each DiT block is initialized as an identity block (for adaLN-zero).
- Final layer is also initialized as zero.

# The OG DiT - the adaLN voodoo



- adaLN-zero is important and is compute-efficient.
- Later works have also explored sharing the adaLN parameters across multiple blocks, reducing compute.
- Conditioning modulation is paramount.

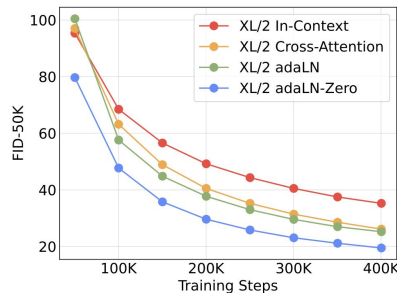
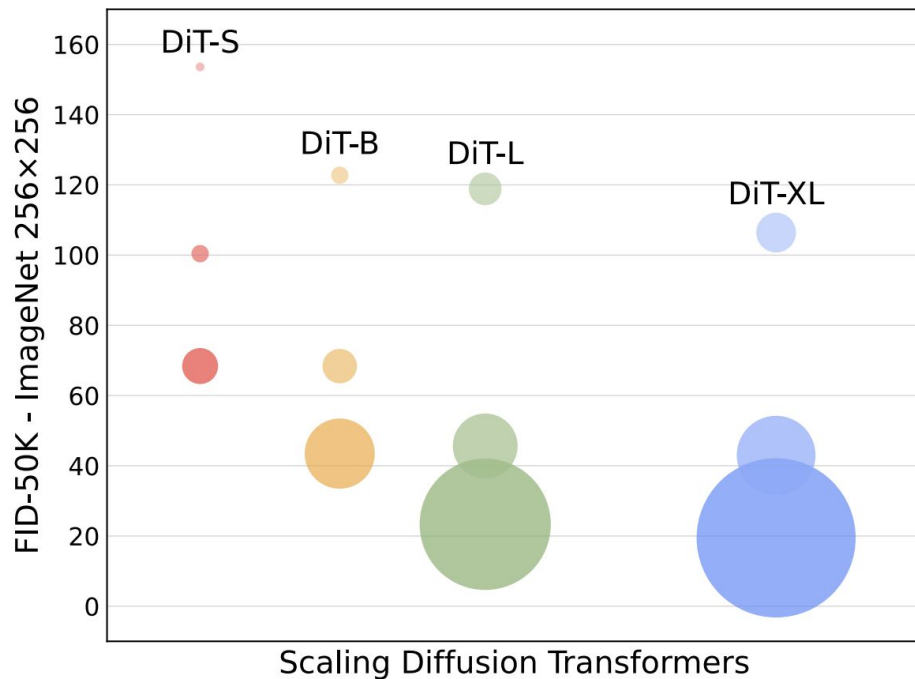


Figure 5. **Comparing different conditioning strategies.** adaLN-Zero outperforms cross-attention and in-context conditioning at all stages of training.

💡 Cross-attention didn't help.

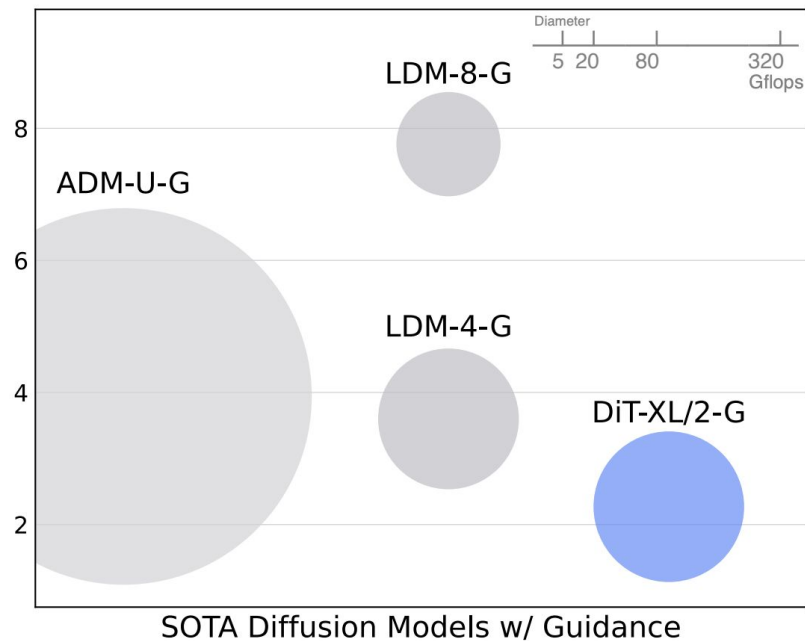
# The OG DiT - results

As expected, it scales gracefully with more compute:



# The OG DiT - results

It is also efficient:

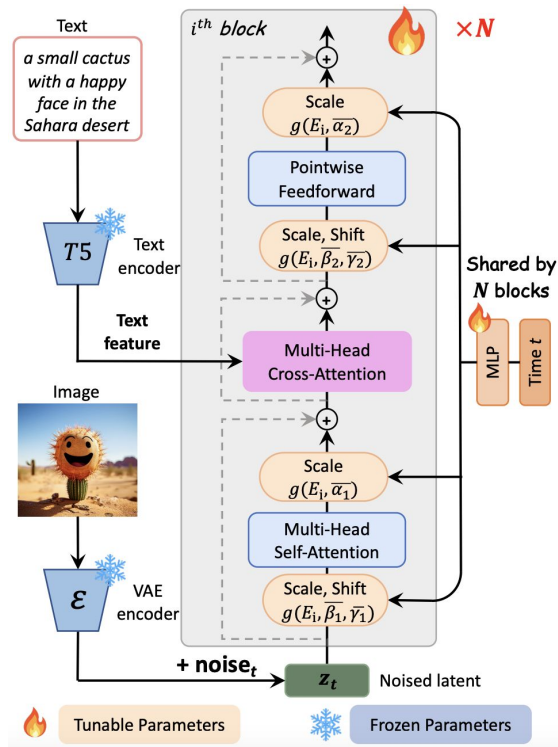


But no one does class-conditional anymore. Show text-to-image!

# Enabling T2I in DiT - PixArt-α

- How to embed the input prompts?
  - Use a text encoder.
- How to learn the contextual dependencies? We have text + timestep.
  - Self-attention on the noisy latents.
  - Use cross-attention between noisy latents and text.
  - Modulate just the timestep.
- Initialize from a class-conditional counterpart if available and compatible.

# Enabling T2I in DiT - PixArt-α



- How to embed the input prompts?
  - Use a text encoder.
- How to learn the contextual dependencies? We have text + timestep.
  - Self-attention on the noisy latents.
  - Use cross-attention between noisy latents and text.
  - Modulate just the timestep.
- Initialize from a class-conditional counterpart if available and compatible.

[PixArt-α: Fast Training of Diffusion Transformer for Photorealistic Text-to-Image Synthesis](#)

# PixArt-α - text encoder

- Flan-T5-xxl to enable long prompts and provide rich text understanding.
- Exploring the space of text encoders is still an interesting area.
  - Integration of regular LLMs isn't very difficult (SANA, Lumina2).
- Prompt embedding is one time and these models are usually memory-bound, so a heavy model is okay.

Photorealistic Text-to-Image Diffusion Models with Deep Language Understanding  
SANA: Efficient High-Resolution Image Synthesis with Linear Diffusion Transformers  
Lumina-Image 2.0: A Unified and Efficient Image Generative Framework

# PixArt-α - return of adaLN voodoo

```
1 adaLN = nn.Sequential(  
2     nn.SiLU(),  
3     nn.Linear(hidden_size, 6 * hidden_size, bias=True)  
4 )  
5 gamma_msa, beta_msa, alpha_msa, gamma_mlp, beta_mlp, alpha_mlp = (  
6     adaLN(c).chunk(6, dim=1)  
7 )
```

For *each DiT block* ( $c = t + y$ )

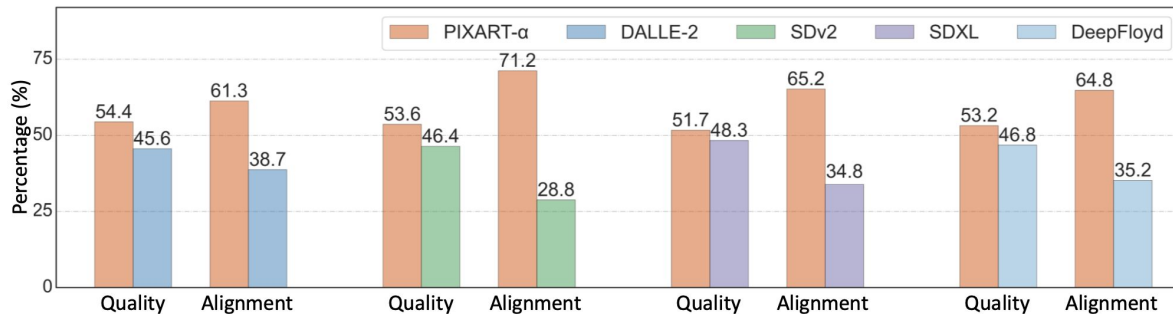
## PixArt-α - return of adaLN voodoo

```
1 t = t.reshape(B, 6, -1)
2 gamma_msa, beta_msa, alpha_msa, gamma_mlp, beta_mlp, alpha_mlp = (
3     self.scale_shift_emb[None] + t).chunk(6, dim=1)
4 )
```

- Before a block,  $t = \text{mlp}(\text{timestep\_embedding}(t))$ . *One-time*.
- For **each PixArt block**, we have component-wise addition with the embeddings. No matmuls.
  - `scale_shift_emb = nn.Parameter(torch.randn(6, hidden_size))`
- No combining of  $c$  and  $t$ . Makes sense as we now have a way to compute embeddings of the prompts.
- Reduces ~27% computation.

# PixArt- $\alpha$ - impressive results for a compact model

| Method           | Type | #Params | #Images | FID-30K↓ |
|------------------|------|---------|---------|----------|
| DALL·E           | Diff | 12.0B   | 250M    | 27.50    |
| GLIDE            | Diff | 5.0B    | 250M    | 12.24    |
| LDM              | Diff | 1.4B    | 400M    | 12.64    |
| DALL·E 2         | Diff | 6.5B    | 650M    | 10.39    |
| SDv1.5           | Diff | 0.9B    | 2000M   | 9.62     |
| GigaGAN          | GAN  | 0.9B    | 2700M   | 9.09     |
| Imagen           | Diff | 3.0B    | 860M    | 7.27     |
| RAPHAEL          | Diff | 3.0B    | 5000M+  | 6.61     |
| PIXART- $\alpha$ | Diff | 0.6B    | 25M     | 7.32     |

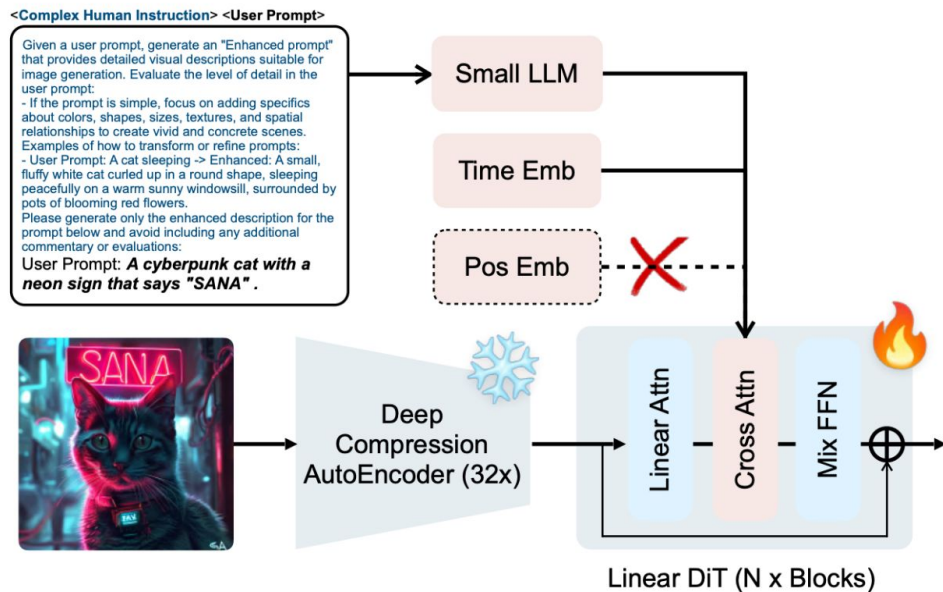


# Return of quadratic time & memory complexity

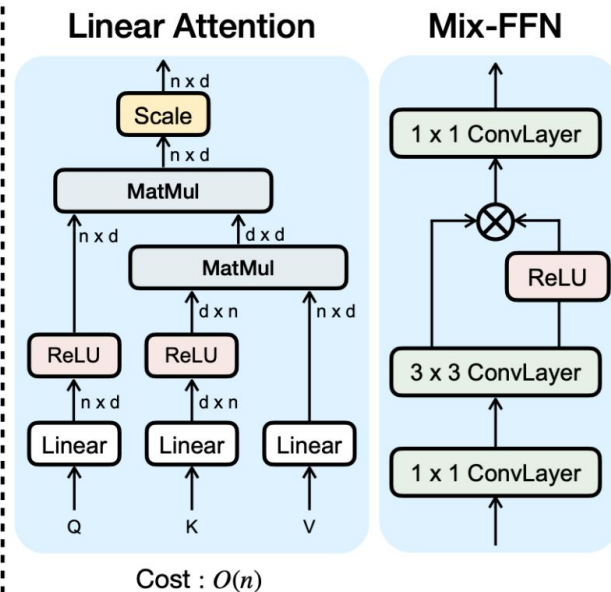
- When targeting ultra high-res (4k), vanilla attention becomes prohibitive even in the latent space e.g., (1, 16, 512, 512).
- Memory (fp16, head\_dim: 128, num\_heads: 24):
  - Each projection: (1, 65536, 3072)
  - $B = 1, N = 65536, H = 24$
  - $B \times H \times N^2 \times 2 \approx \mathbf{190GB!}$  (Flash attention of course helps)\*
- What can we do?
  - Operate on an even more compressed latent space?
  - Use some form of linear attention?

\*Script to calculate [this](#).

# SANA - efficient DiT for T2I



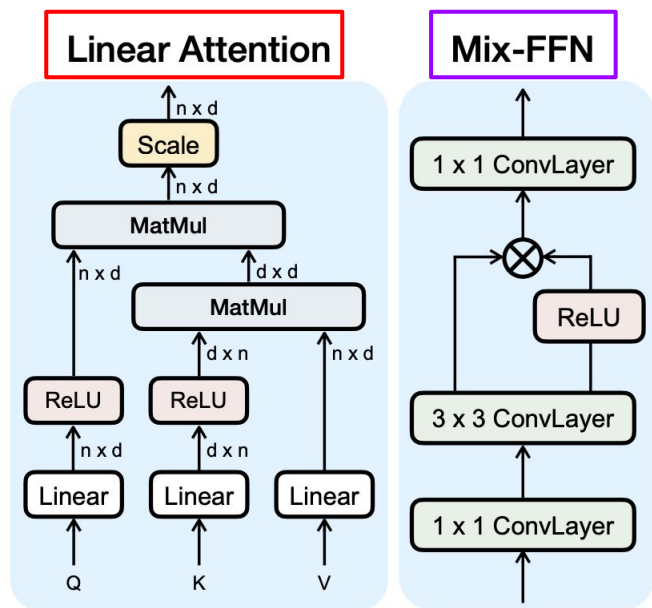
(a). Architecture overview of our Sana.



(b). Linear DiT Module.

[SANA: Efficient High-Resolution Image Synthesis with Linear Diffusion Transformers](#)

# SANA - efficient DiT for T2I



Cost :  $O(n)$

(b). Linear DiT Module.

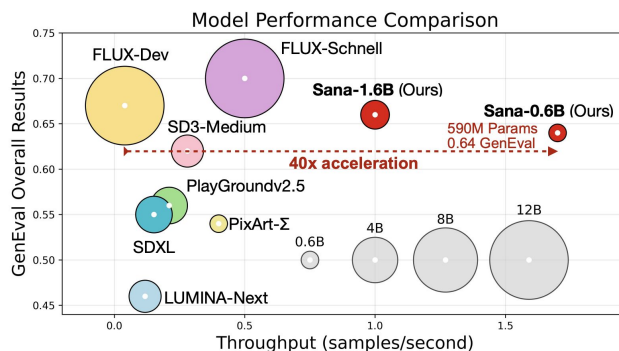
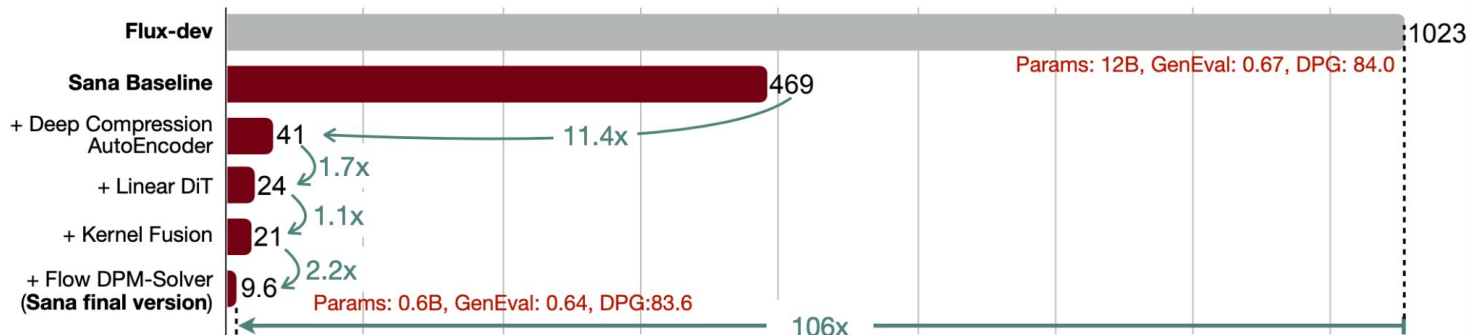
- Self-(linear)attention + cross-attention.
- No  $N^2$  computation in self-attention:

$$O_i = \sum_{j=1}^N \frac{\text{ReLU}(Q_i) \text{ReLU}(K_j)^T V_j}{\sum_{j=1}^N \text{ReLU}(Q_i) \text{ReLU}(K_j)^T} = \frac{\text{ReLU}(Q_i) \left( \sum_{j=1}^N \text{ReLU}(K_j)^T V_j \right)}{\text{ReLU}(Q_i) \left( \sum_{j=1}^N \text{ReLU}(K_j)^T \right)}$$

- ReLU instead of softmax.
- Mix-FFN block to model local interactions.
  - Also helps with NoPE

EfficientViT: Memory Efficient Vision Transformer with Cascaded Group Attention  
 SegFormer: Simple and Efficient Design for Semantic Segmentation with Transformers

# SANA - impressive performance x speed tradeoff



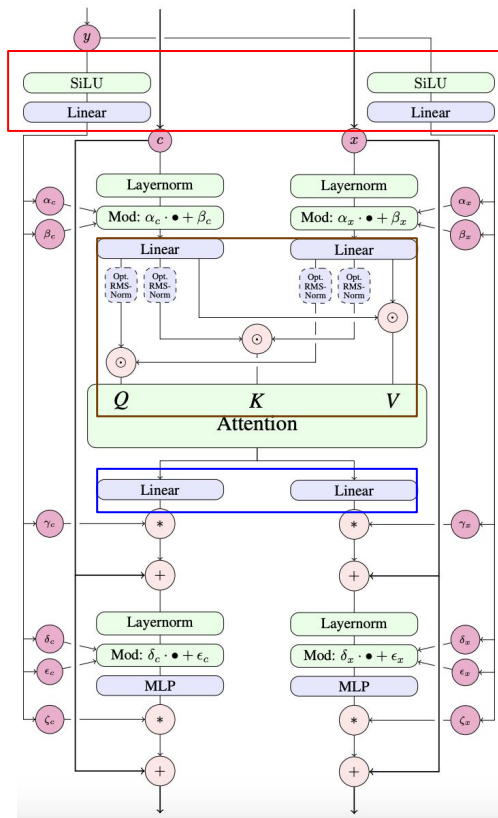
## Multi-modality in separate spaces – MMDDiT (SD3)

- Space of text embeddings can be different and end up introducing biases (unidirectionality).
- How about we do QKV separately on the text and image latents separately and then concatenate the outputs before attention?
- Conceptually:

```
1 i_q, i_k, i_v = qkv(img_latents).chunk(3, dim=-1)
2 c_q, c_k, c_v = context_qkv(txt_embs).chunk(3, dim=-1)
3
4 q = torch.cat([i_q, c_q])
5 k = torch.cat([i_k, c_k])
6 v = torch.cat([i_v, c_v])
7
8 # rest of the stuff
9 ...
```

# Multi-modality in separate spaces – MMDiT (SD3)

[Scaling Rectified Flow Transformers for High-Resolution Image Synthesis](#)



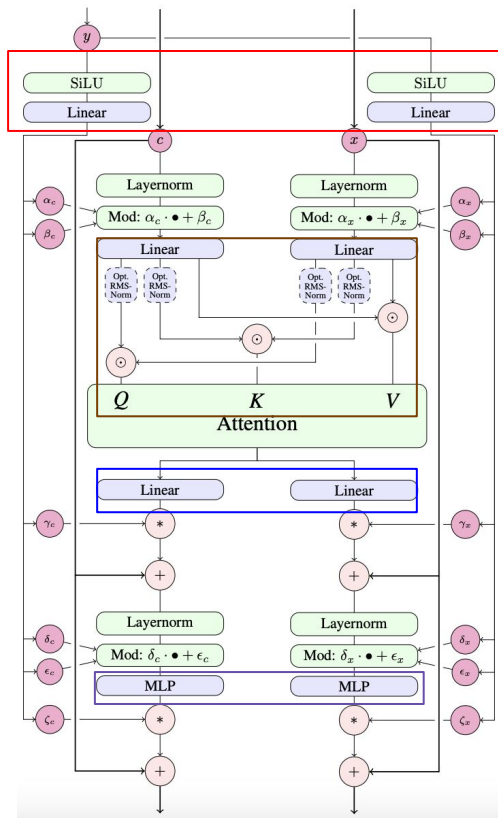
Separate  $\text{adaLN}$

Separate  $\text{QKV}$

Separate out proj

# Multi-modality in separate spaces – MMDiT (SD3)

[Scaling Rectified Flow Transformers for High-Resolution Image Synthesis](#)



Separate adaLN

Separate QKV

Separate out proj

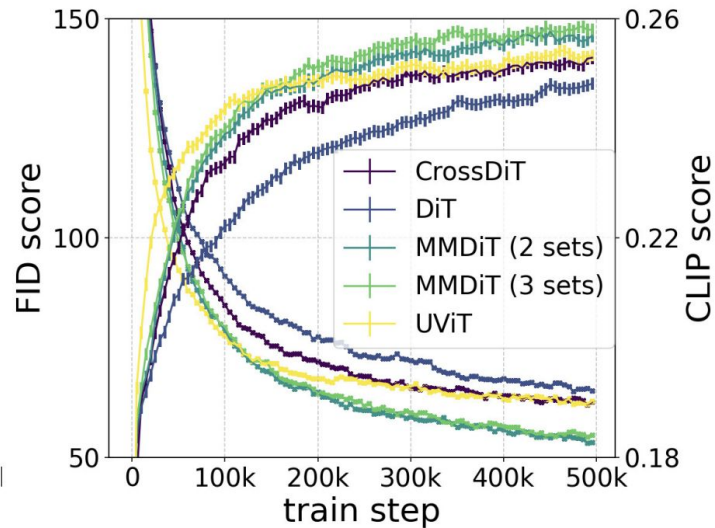
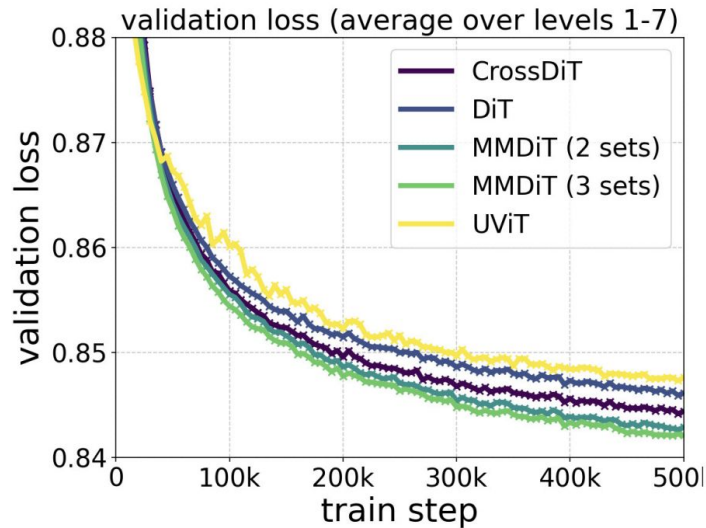
Separate FF

## Multi-modality in separate spaces – MMDiT (SD3)

- Gives a way to co-evolve different modalities for the given task.
- Concatenation before attention gives a way for cross-modal interaction.  
*How do we do masking then?*
- Modulation happens with a both timestep embeddings and pooled text embeddings.
- Uses **three** text encoders (2 CLIPs, 1 T5-xxl)! But they can be mix-matched during inference.

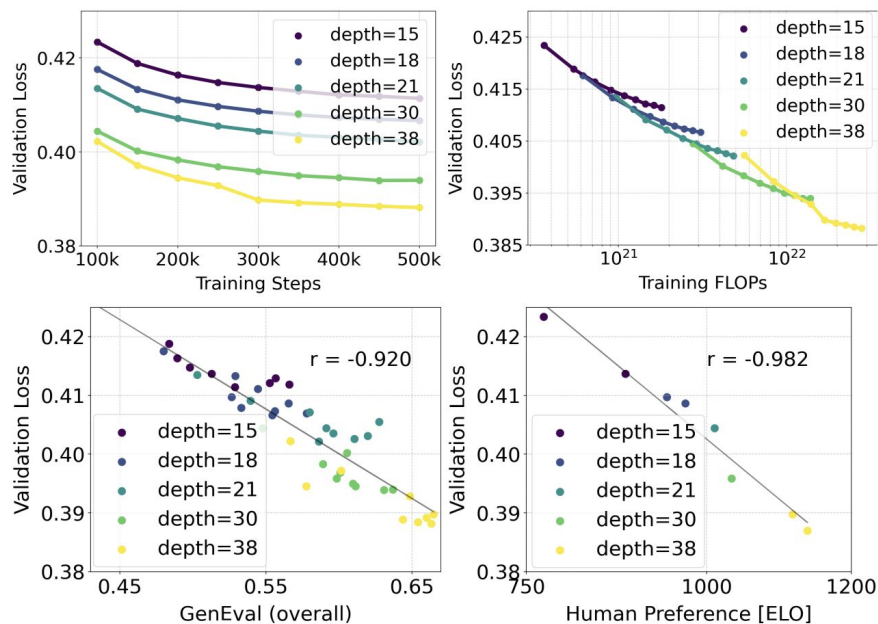
# MMDiT - does it matter?

The plots highly suggest so:



# MMDiT - scaling benefits

Scales very well but needs QK-normalization



Scaling Vision Transformers to 22 Billion Parameters

MMDiT / SD3 is incomplete without this 🦛



A whimsical and creative image depicting a hybrid creature that is a mix of a waffle and a hippopotamus. This imaginative creature features the distinctive, bulky body of a hippo, but with a texture and appearance resembling a golden-brown, crispy waffle. The creature might have elements like waffle squares across its skin and a syrup-like sheen. It's set in a surreal environment that playfully combines a natural water habitat of a hippo with elements of a breakfast table setting, possibly including oversized utensils or plates in the background. The image should evoke a sense of playful absurdity and culinary fantasy.

# Flavors of MMDDiT

- Do we need all blocks to be MMDDiT? It's computationally demanding.
- Combine MMDDiT and regular DiT blocks to better optimize FLOPs.
- Conceptually:

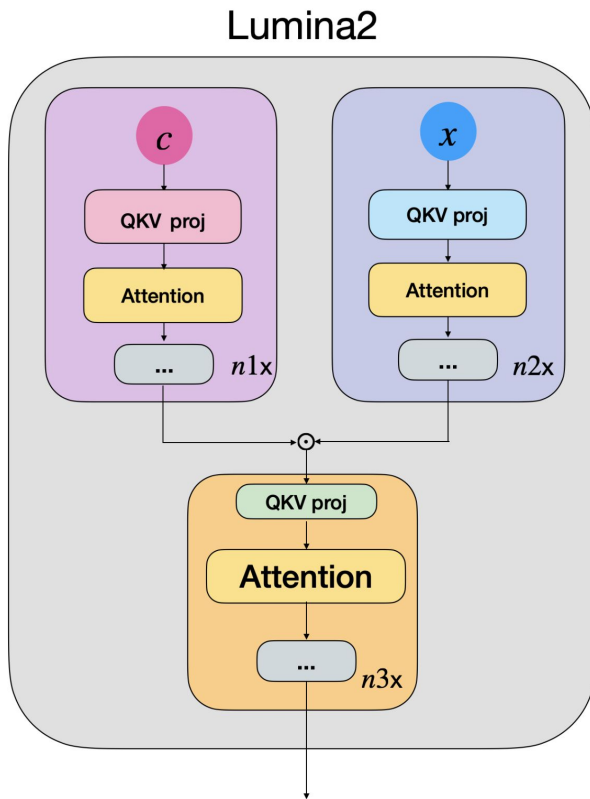
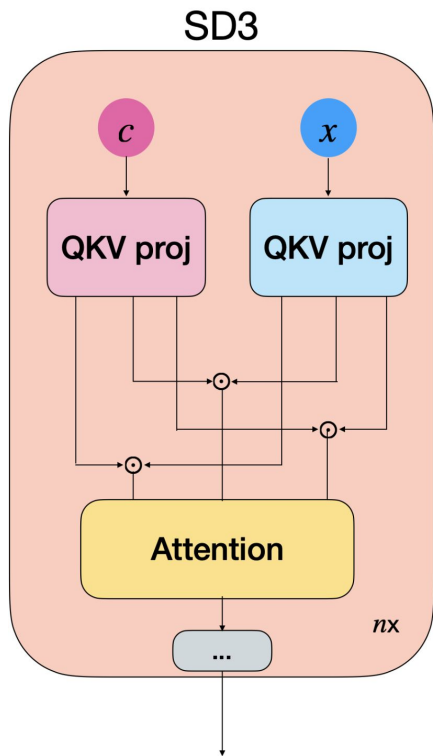
```
1 # Regular MMDDiT blocks.
2 for block in self.double_blocks:
3     context, x = block(
4         x=x,
5         context=context,
6         ...
7     )
8
9 # Concatenate.
10 context_x = torch.cat([context, x], dim=1)
11
12 # Continue with the rest.
13 for block in self.single_blocks:
14     x = block(x=context_x, ...)
```

Popular examples: [Flux](#), [AuraFlow](#).

# Flavors of MMDiT

- Another twisty one:
  - Attention on `modality_A` => `output_A`.
  - Attention on `modality_B` => `output_B`.
  - Concatenate `output_A`, `output_B` => `output_AB`.
  - Attention on `output_AB`.
- Configs of blocks for both modalities and concatenated outputs can vary for more flexibility.

# Flavors of MMDiT

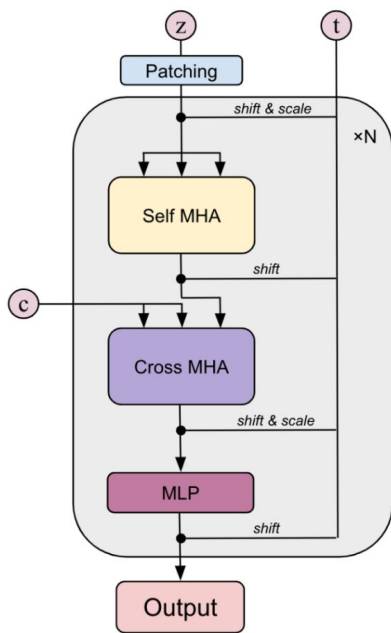


[Lumina-Image 2.0: A Unified and Efficient Image Generative Framework](#)

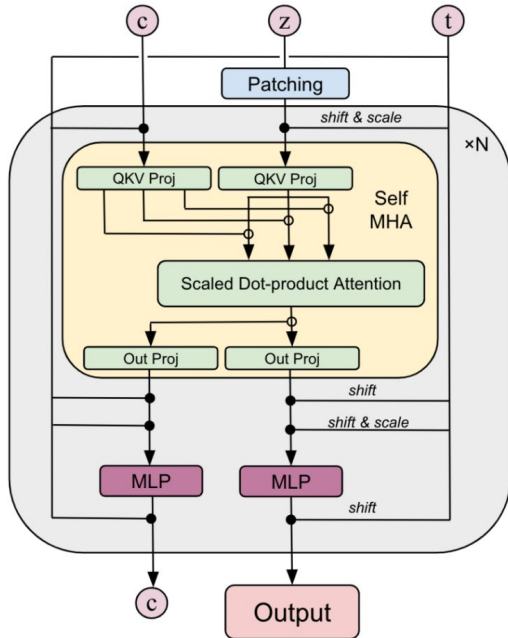
# Enabling T2I in DiT - simplifying the design

- How much is parameter sharing helpful?
  - We saw adaLN.
  - Can we share QKVO and MLP too?
- Do we need to do self-attention and then cross-attention for T2I or even MMDiT-like?
  - Project on `torch.cat([img_tokens, txt_tokens])` and then self-attention?
- Text encoders
  - SD3 has three (CLIP x2, T5), PixArt has just one (T5), Flux has two (CLIP, T5), SANA has one (Gemma) !

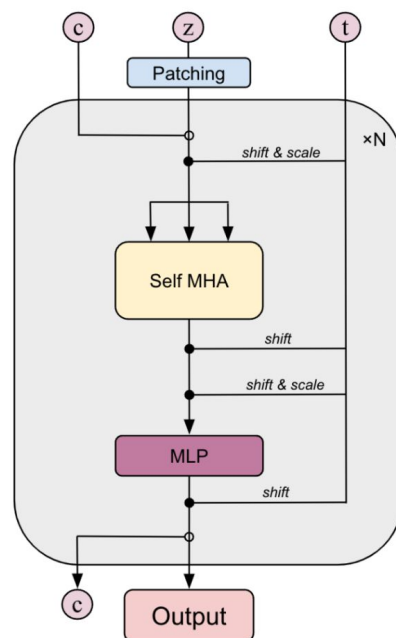
# Simplifying the design in T2I DiT - DiT-Air



(a) PixArt- $\alpha$



(b) MMDiT

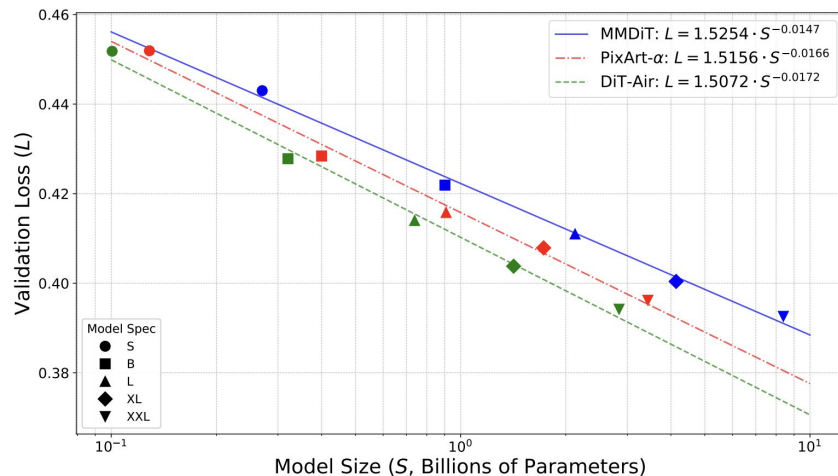


(c) DiT-Air

# Simplifying the design in T2I DiT - DiT-Air

- Share adaLN params like PixArt.
- Concatenate image and text tokens and operate with a single set of projections (QKVO and MLP).

| Model           | Params | Val. ↓ | FID ↓ | CLIP ↑ | Pick ↑ | GenE. ↑ | Aesth. ↑ | T2I. ↑ |
|-----------------|--------|--------|-------|--------|--------|---------|----------|--------|
| Per-layer AdaLN | 902M   | 0.422  | 14.7  | 32.9   | 20.28  | 69.8    | 5.62     | 51.1   |
| Shared AdaLN    | 631M   | 0.422  | 15.0  | 32.9   | 20.28  | 69.8    | 5.59     | 51.0   |



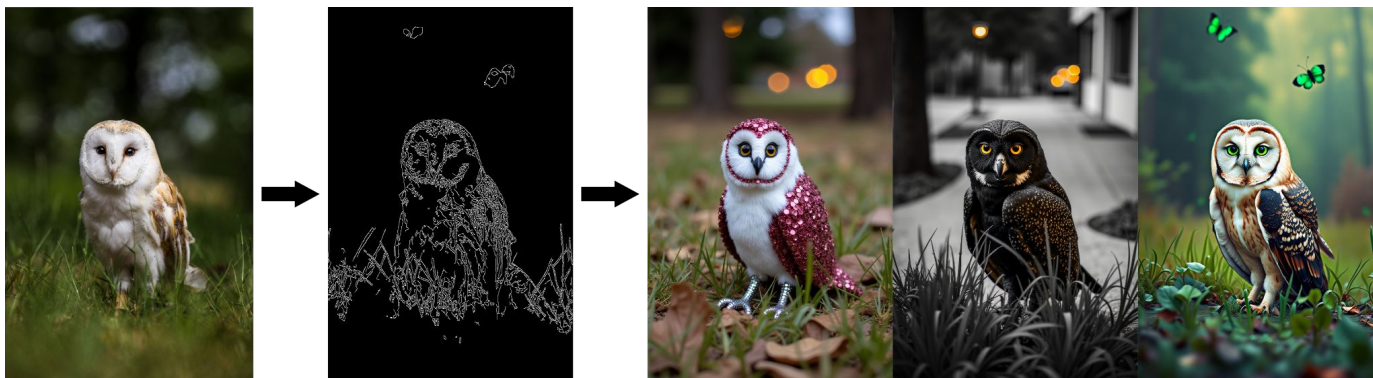
# Simplifying the design in T2I DiT - DiT-Air

- Share QKVO and MLP params (both or just QKVO) compromising quality.
- Bidirectional CLIP + text-only LLM is better for text representations.
- Post-training with SFT and reward-tuning.

| Model                | Size (B) |           | Metrics     |             |             |             |             |                   |
|----------------------|----------|-----------|-------------|-------------|-------------|-------------|-------------|-------------------|
|                      | Total    | Trainable | FID ↓       | CLIP ↑      | Pick ↑      | GenE. ↑     | Aesth. ↑    | T2I. ↑            |
| SDXL Base [34]       | 3.5      | 2.6       | 268.0       | 22.1        | 17.0        | 55.0        | 4.32        | 40.6              |
| PixArt- $\alpha$ [4] | 5.4      | 0.6       | 120.7       | 27.3        | 17.0        | 55.7        | 5.76        | 44.7              |
| SD3 Medium [14]      | 7.7      | 2.0       | 26.0        | 32.0        | 20.7        | 62.0        | 5.99        | 52.4              |
| SD3 [14]             | 13.6     | 8.0       | –           | –           | –           | 74.5        | –           | 51.4 <sup>†</sup> |
| Flux-Dev [26]        | 16.9     | 12.0      | 68.7        | 30.2        | 19.7        | 66.7        | 6.12        | 49.6              |
| Flux-Schnell [26]    | 16.9     | 12.0      | 25.1        | 33.1        | 21.6        | 70.7        | 6.12        | 49.9              |
| JanusPro-7B [7]      | 6.9      | 6.9       | <b>17.2</b> | 15.5        | 16.6        | 80.3        | 5.95        | 35.2              |
| DiT-Air/L-Lite       | 1.2      | 0.7       | 23.1        | 33.9        | 21.5        | 78.4        | 6.06        | 55.4              |
| DiT-Air/XXL          | 6.0      | 2.8       | 32.2        | <b>34.7</b> | <b>22.1</b> | <b>82.9</b> | <b>6.29</b> | <b>59.5</b>       |

## Other structural controls?

- T2I is cool. What about conditioning with structural signals too (e.g., scribbles, poses)?
- Can we combine multiple structural signals to better control the outputs?



[black-forest-labs/FLUX.1-Canny-dev](https://black-forest-labs.com/FLUX.1-Canny-dev)

## Other structural controls?

- Could use an auxiliary network to model the visual signals and find a way to inject in the base DiT model ([ControlNet](#), [PixArt- \$\delta\$](#) ).
- Change the base DiT model
  - Increase the input channels of the base DiT to enable additional inputs ([FluxControl](#)).
  - A small adapter to enable unified processing of noisy latents and condition tokens ([OminiControl](#)).
- Structural control has spatial correspondence. Other forms of control is also important (non-spatial; subject-driven generation, edits, etc.).

# Videos?

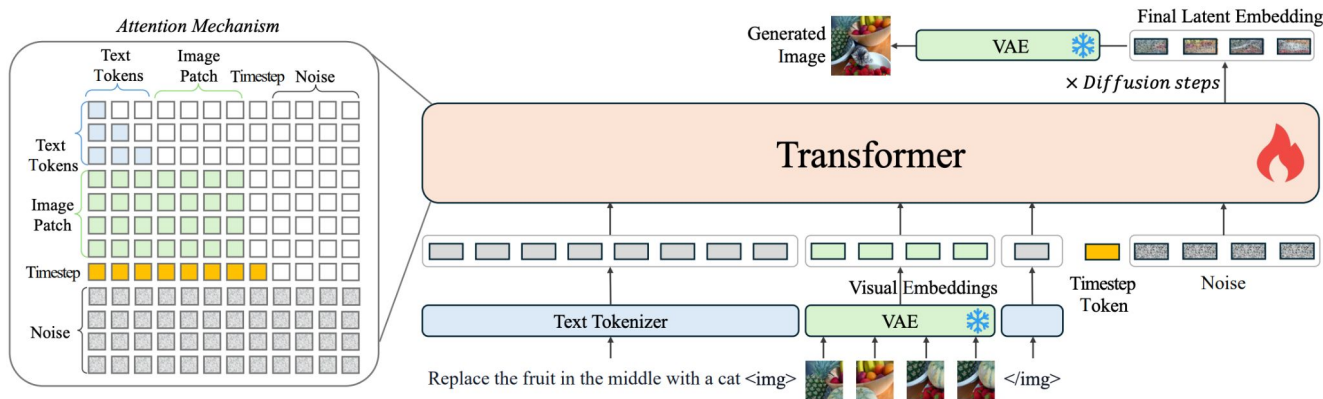
- RoPE for positional encoding.
- Full 3D attention > factorized attention for quality.
  - Both approaches self + cross attention, MMDiT-style work but come with trade-offs.
- Efficiency: [LTX-Video](#)
- Performance: [Wan](#), [HunyuanVideo](#)



[spacepxl/Wan2.1-control-loras](#)

# Next-gen architectures

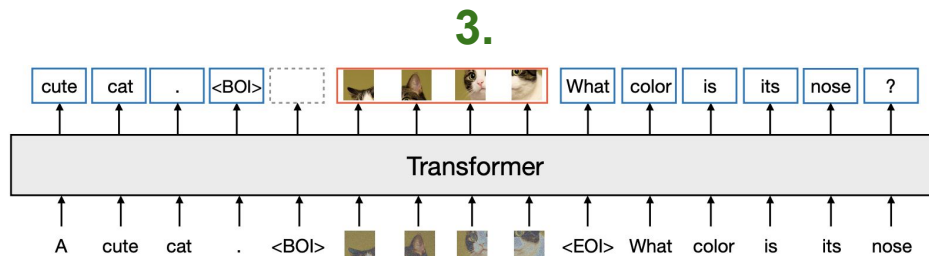
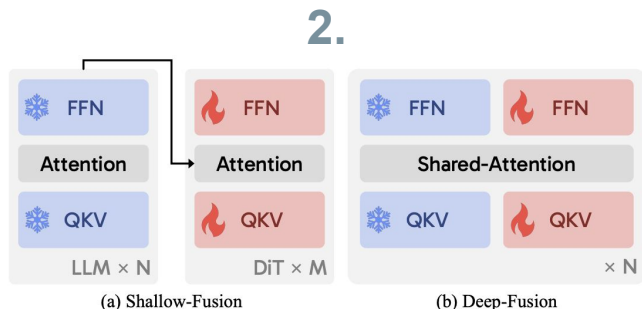
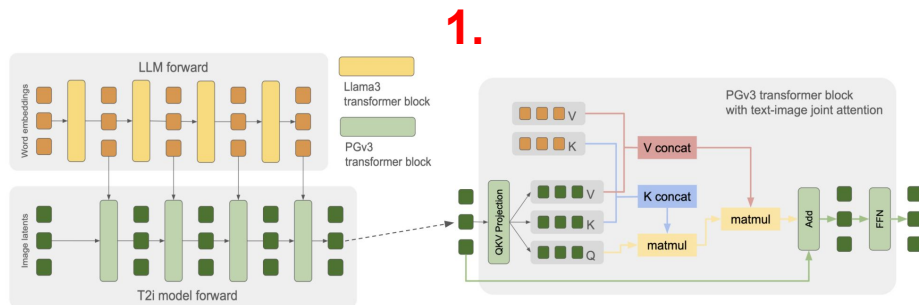
- What about in-context learning?
- Current architectures aren't efficient enough to enable it in a reasonable time.
- Start from a pre-trained LLM and enable continuous modality.



[OmniGen: Unified Image Generation](#)

# Next-gen architectures

1. Playground v3
2. FuseDiT (CVPR'25)
3. Transfusion



The only one that is open-sourced :)

# Implementations for hacking and getting started

We provide reasonably clean implementations of many of these architectures in Diffusers. Explore them!

<https://github.com/huggingface/diffusers>

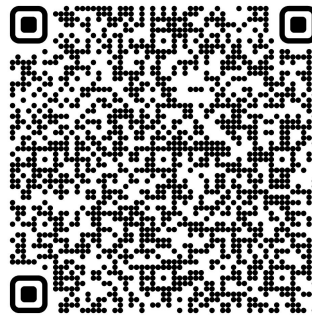
```
1 from diffusers import DiffusionPipeline
2 import torch
3
4 pipe = DiffusionPipeline.from_pretrained(
5     "black-forest-labs/FLUX.1-dev", torch_dtype=torch.bfloat16
6 ).to("cuda")
7 image = pipe("A tiny astronaut hatching from an egg on the moon").images[0]
8 image.save("astronaut.png")
```

# Concluding thoughts

- Introduction of MoEs in the mix ([HiDream](#)).
- Couldn't cover training much (it's a different beast, maybe CS'25 next edition 😊).
  - Data shenanigans, multi-stage training vs. single-stage, representation alignment, post-training, etc.
- These architectures go well beyond image and video generation
  - Robotics ([lerobot/diffusion\\_pusht](#))
  - Gene synthesis
- Interpretation of these things are non-trivial.
- Performs very well low-level computer vision tasks too ([diffusion-vision.github.io/](#)).



**Flux.1 Dev prompt:** Realistic photo of a puppy hatching from an egg on the moon and holding a placard saying "Thank you!"



Slides: [bit.ly/dit-cs25](https://bit.ly/dit-cs25)